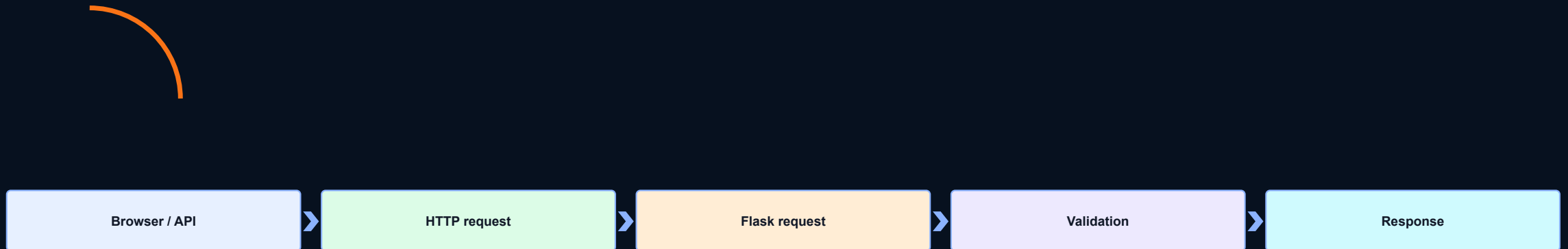


Request Handling & User Input

Request object, args, form, JSON, methods and basic validation

Core idea

A browser or API client sends input. Flask collects it in request. Your code validates and responds safely.



Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

Today's learning map

By the end of class, students should confidently read input from URLs, forms and JSON bodies.



Request object

The central object that stores input sent by the client.

request.args

Read query string values from a GET URL.

request.form

Read HTML form fields submitted using POST.

request.get_json()

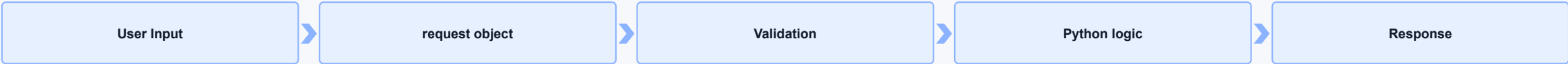
Read JSON body data from API clients like Postman.

request.method

Detect GET, POST, PUT or DELETE in a route.

Basic validation

Check required values, type, length and safe input.



Class outcome

Build a small Feedback Collector app that accepts search input, form input and JSON input without crashing.

Quick recall from Day 4

Day 5 starts where URL building and HTTP methods ended.



```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        # form will be processed here
        pass
    return render_template('register.html')
```

URL building

url_for() generates safe URLs from endpoint names.

Redirect

redirect() sends the browser to another route.

GET

Usually reads or displays a page.

POST

Usually submits data to the server.



Transition question

When a form is submitted, where exactly does Flask keep the input values? Answer: inside the request object.

What is the request object?

request is Flask's way of giving your route access to the current incoming HTTP request.



Simple definition

request stores what the client sent: method, URL data, form fields, files, headers, cookies and JSON body.

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/info')
def info():
    return request.method
```

Key import

from flask import request

Important rule

request belongs to the current request only.

HTTP request



Flask creates request



Route uses request



Route returns response

Trainer prompt

Ask students: Can the same route behave differently for different users? Yes, because each request can carry different input.

Where can user input come from?



Different types of input live in different parts of the request object.

Input source	Flask property	Example
URL query	request.args	/search?city=hyd
HTML form	request.form	name, email, password
JSON body	request.get_json()	{name: 'Asha'}
HTTP method	request.method	GET or POST
Uploaded file	request.files	profile image, resume

Exam-friendly memory line

args = address bar input
form = HTML form input
JSON = API body input

Classroom note

Do not mix up request.args and request.form. They answer different questions.

request.args: reading query parameters

Use request.args when data is sent in the URL after a question mark.



```
from flask import Flask, request

app = Flask(__name__)

@app.route('/search')
def search():
    keyword = request.args.get('q')
    return f'Searching for: {keyword}'
```

URL

/search?q=python

Key

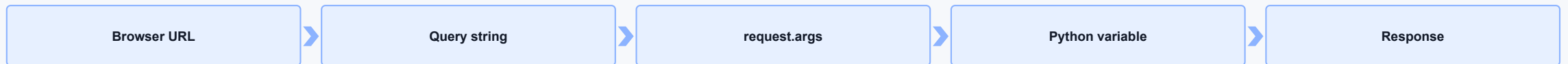
q

Value

python

Output

Searching for: python



Best practice

Use .get() instead of square brackets to avoid KeyError when the user does not provide a value.

Live demo: search input using GET

Create a page where students search for a course using the address bar or a small form.



```
@app.route('/course-search')
def course_search():
    course = request.args.get('course', '')

    if course:
        return f'Course searched: {course}'

    return 'Please search a course'
```

Try these URLs

/course-search
/course-search?course=flask
/course-search?course=python

Case 1

No query parameter -> show default message.

Case 2

course value exists -> show selected course.

Student challenge

Add city and mode as extra query parameters: /course-search?course=flask&city=hyd&mode=offline

Acceptance check

The route must not crash when the parameter is missing.

request.form: reading HTML form data

Use request.form when the browser submits an HTML form using POST.

```
@app.route('/feedback', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        name = request.form.get('name')
        rating = request.form.get('rating')
        return f'Thank you {name}. Rating: {rating}'

    return render_template('feedback.html')
```

```
<form method="POST">
  <input name="name">
  <input name="rating">
  <button type="submit">Send</button>
</form>
```



Memory trick

GET form vs POST form

Both can send input, but they are used for different purposes.



Method	Common use	Where data appears	Read using
GET	Search, filters, public data	Visible in URL	request.args
POST	Register, login, feedback, sensitive input	Hidden in request body	request.form

GET example

/products?category=mobile&brand=samsung

POST example

Login form sends email and password in request body.

Exam trap

Do not say GET is always unsafe or POST is automatically secure. HTTPS is what encrypts traffic.

request.get_json(): reading API input

Use JSON body input when building APIs or testing with Postman.



```
@app.route('/api/feedback', methods=['POST'])
def api_feedback():
    data = request.get_json()

    name = data.get('name')
    message = data.get('message')

    return {'name': name, 'message': message}
```

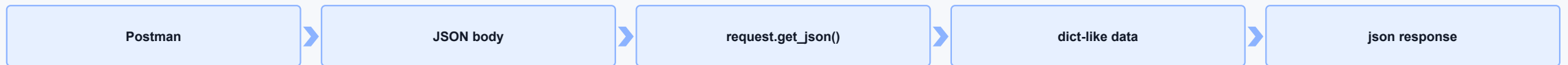
```
{
  "name": "Ravi",
  "message": "Flask is clear"
}
```

Client

Postman or frontend app

Header

Content-Type: application/json



Common issue

request.method: deciding what to do

A single route can display a form on GET and process it on POST.

```
@app.route('/contact', methods=['GET', 'POST'])
def contact():
    if request.method == 'POST':
        name = request.form.get('name')
        return f'Message received from {name}'

    return render_template('contact.html')
```



GET request

Show form



POST request

Process form

GET

Read or display

POST

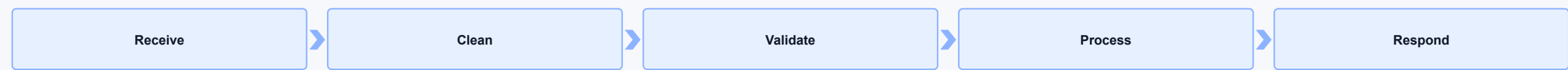
Submit or create

Trainer prompt

Ask: Why should the same URL support both GET and POST for forms?

Handling user input: safe thinking

User input is never trusted blindly. Always read, clean, validate and then process.



Receive Read value using args, form or JSON.	Clean Use .strip() to remove spaces.	Validate Check required, length, type and range.	Process Save, search, calculate or return data.	Respond Give useful success or error message.
--	--	--	---	---

```
name = request.form.get('name', '').strip()
if not name:
    return 'Name is required'
```

Golden rule Do validation before database insert, login check, API response or file operation.

Basic validation patterns

Start with simple checks. Later these same ideas become production validation rules.



Validation	Code idea	Why it matters
Required field	if not name:	Prevents empty submission
Trim spaces	name.strip()	Removes accidental leading/trailing spaces
Length check	len(password) >= 8	Avoids weak or incomplete data
Type check	age.isdigit()	Confirms numeric input before conversion
Range check	1 <= rating <= 5	Keeps value in allowed limits
Email check	'@' in email	Basic beginner-level email validation

Important

Validation is not only for exams. Every real Flask project needs validation before accepting user input.

Live demo: validate a feedback form

Students should see both success and error paths.



Test 1

Submit empty name

Test 2

Submit abc as rating

Test 3

Submit rating 10

Test 4

Submit valid data

Discussion

Why convert rating to int only after isdigit()? Because `int("abc")` would crash.

```
@app.route('/feedback', methods=['GET', 'POST'])
def feedback():
    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        rating = request.form.get('rating', '').strip()

        if not name:
            return 'Name is required'
        if not rating.isdigit():
            return 'Rating must be a number'

        rating = int(rating)
        if rating < 1 or rating > 5:
            return 'Rating must be from 1 to 5'

        return f'Thanks {name}! Rating: {rating}'
```

Class checkpoint

Mini project: Student Feedback Collector

This is the Day 5 integration task. It combines args, form, JSON, method and validation.



Route 1: /

Show project home page and links to form/search/API instructions.

Route 2: /search

Use request.args to search by student name or course.

Route 3: /feedback

Use GET to show form and POST to process form.

Route 4: /api/feedback

Use request.get_json() to accept JSON from Postman.

Validation

Name required, email format, rating 1-5, message length.

Output

Return success message or clear error message.



Submission evidence

Project structure for Day 5

Keep the project simple and organized.



```
day5_feedback_app/  
├── app.py  
└── templates/  
    ├── home.html  
    └── feedback.html
```

app.py

Routes, request handling and validation logic.

home.html

Landing page with links and instructions.

feedback.html

HTML form with name, email, rating and message fields.

Postman

Used to test JSON route.

```
python -m venv .venv  
.venv\Scripts\activate  
pip install flask  
python app.py
```

Common errors to catch quickly

These are the mistakes students are most likely to make on Day 5.



Missing import

NameError: request is not defined
Fix: Use: from flask import request

Wrong method list

POST route gives 405
Fix: Add methods=['GET','POST']

Wrong field name

None value from form
Fix: Match input name exactly

Unsafe int()

ValueError on text input
Fix: Use isdigit() before int()

Missing default

NoneType error
Fix: Use .get('key', '')

Wrong JSON header

JSON body not read
Fix: Use application/json in Postman

Debugging habit

Security and best practices

Beginner validation is the first step toward secure Flask development.



Do not trust input

Users can send empty, wrong, extra or malicious values.

Use defaults safely

`request.form.get('name', '')` avoids None problems.

Validate before save

Never insert invalid input into the database.

Return clear errors

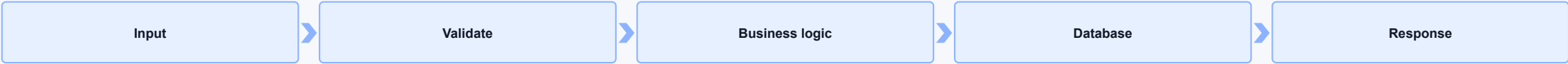
Students should understand why input failed.

Avoid sensitive GET data

Passwords should never be sent in query strings.

Prepare for next project

Registration, login and CRUD all depend on request handling.



Classroom practice plan

A practical 45-60 minute flow after the concept explanation.



Round 1 - args

Build `/search?name=...` using `request.args`.

Round 2 - form

Build `/feedback` with GET form and POST submit.

Round 3 - JSON

Build `/api/feedback` and test using Postman.

Round 4 - validation

Add required, numeric and range checks.

Round 5 - debug

Break field names and methods intentionally.

Round 6 - submit

Push code with screenshots and short README.

Assessment evidence

A student is ready if they can explain which request property is used for each input type.

Day 5 quick quiz

Use this as a class exit check before giving the homework.



1. Which object stores incoming request data in Flask?
2. Which property reads ?course=flask from the URL?
3. Which property reads HTML form data submitted by POST?
4. Which method reads a JSON body from Postman?
5. Why do we use request.method?
6. Why is .get() safer than square brackets?
7. Why should int() be used after isdigit()?
8. Give one example of basic validation.

```
request.args  
request.form  
request.get_json()
```

Trainer instruction

Ask students to answer orally first, then write the answers in their notebook.

Pass mark

Student should answer at least 6 out of 8 confidently.

Homework assignment

Complete and submit before the next Flask class.



Task 1 - Search route

Create `/student-search?name=...` and display a friendly result.

Task 2 - Feedback form

Create a POST form with name, email, rating and message.

Task 3 - Validation

Name required, email contains `@`, rating must be 1 to 5, message minimum 10 chars.

Task 4 - JSON API

Create `/api/feedback` and test valid and invalid data in Postman.

Task 5 - README

Write how to run the app and include sample URLs and JSON body.

Task 6 - Screenshots

Add screenshots of form success, validation error and Postman output.

Tomorrow connection

Day 6 starts the Student Notes Manager project. Request handling becomes the base for registration and login.

Day 5 recap

Students should leave with these five points fixed in memory.



1. request is used to access incoming HTTP request data.
2. request.args reads URL query parameters.
3. request.form reads submitted HTML form fields.
4. request.get_json() reads API JSON body data.
5. request.method helps one route handle GET and POST differently.
6. Validation protects the app from empty, wrong or unsafe input.

```
name = request.form.get('name',  
'').strip()  
if not name:  
    return 'Name required'
```

Most important question

Where is the input coming from: URL, form or JSON?

Best answer

Choose args, form or get_json based on input source.

Next class: Project Introduction & Database Design